Лабораторна робота №11

Семенюка Владислава Григоровича

Група ІПЗ-21

Варіант 7

Код консольної програми

```
using Lab11Models;



class Program
{
    static void Main()
    {
        const string divider =
"-----------------------------------------------
-------------";

        const int variant = 7;
        var threshold = 5 + variant;
        var last = threshold + 2;

        Console.WriteLine(divider);
        Console.WriteLine("Lab №11");
        Console.WriteLine("Fullname: Vlad
Semeniuk");
        Console.WriteLine("Group: IPZ-21");
        Console.WriteLine($"Variant: {variant}");
        Console.WriteLine($"Last: {last}");
```

```
        Console.WriteLine($"Threshold:
{threshold}");
        Console.WriteLine(divider);

        Console.WriteLine("Delegate
Manipulations:");
        StatusReporter statusReporter = new
StatusReporter(label: "FirstOne");

        ValueHandler? actions =
statusReporter.PrintShort;
        ValueHandler extra = value =>
Console.WriteLine($"Extra: {value}");

        Console.WriteLine("1. Subscribe (PrintShort,
extra(lambda))\n");
        actions += statusReporter.PrintDetailed;
        actions += extra;
        actions?.Invoke(threshold);
        Console.WriteLine("After PrintDetailed
unsubscribe:");
        Console.WriteLine("2. Unsubscribe
(PrintDetails)\n");
        actions -= statusReporter.PrintDetailed;
        actions?.Invoke(threshold);
        Console.WriteLine(divider);

        Console.WriteLine("Event Manipulations:\n");
```

```
        ProgressCounter progressCounter = new
ProgressCounter();

        Console.WriteLine("1. Subscribe (PrintShort,
PrintDetailed, extra(lambda))");
        progressCounter.ThresholdReached +=
statusReporter.PrintShort;
        progressCounter.ThresholdReached +=
statusReporter.PrintDetailed;
        progressCounter.ThresholdReached += extra;
        progressCounter.CountTo(last, threshold);
        Console.WriteLine(divider);

        Console.WriteLine("2. Unsubscribe
(PrintDetailed)");
        progressCounter.ThresholdReached -=
statusReporter.PrintDetailed;
        progressCounter.CountTo(last, threshold);
        Console.WriteLine(divider);

        Console.WriteLine("3. Unsubscribe
(PrintShort, PrintDetailed, extra(lambda))");
        progressCounter.ThresholdReached -=
statusReporter.PrintShort;
        progressCounter.ThresholdReached -= extra;
        progressCounter.CountTo(last, threshold);
        Console.WriteLine("Nothing was printed");
        Console.WriteLine(divider);
```

```
        // Exp. The invoker controls the moment, the
subscribers control
        // the behavior. The delegate acts as a
mediator that connects them
        // without making them know about each
other.

        // Individual assignment A
        Console.WriteLine("1. Subscribe
AuditReporter's static method to Threshold:\n");
        progressCounter.ThresholdReached +=
AuditReporter.PrintAudit;
        progressCounter.CountTo(last, threshold);
        Console.WriteLine("2. Unsubscribe
AuditReporter's static method from Threshold:\n");
        progressCounter.ThresholdReached -=
AuditReporter.PrintAudit;
        progressCounter.CountTo(last, threshold);
        Console.WriteLine("Nothing was printed");
        Console.WriteLine(divider);

        // Epx. It's compatible because the static
PrintAudit method has the same
        // function type, function parameters as our
delegate ValueHandler.
```

```
        // In the first step we got PrintAudit only
printed then we got nothing.



    }
}
```

```
namespace Lab11Models;

public class AuditReporter
{
    public static void PrintAudit(int value)
    {
        Console.WriteLine($"Audit value: {value}");
    }
}
namespace Lab11Models;

public class ProgressCounter
{
    public event ValueHandler? ThresholdReached;


    public void CountTo(int last, int threshold)
```

```
    {
        for (int current = 1; current <= last;
current++)
        {
            if (current == threshold)
            {
                ThresholdReached?.Invoke(current);
            }
        }
    }
}
namespace Lab11Models;

public class StatusReporter
{
    public string Lable { get; }

    public StatusReporter(string label)
    {
        Lable = label;
    }

    public void PrintShort(int value)
    {
        Console.WriteLine($"Value: {value}");
    }


    public void PrintDetailed(int value)
    {
        Console.WriteLine($"Threshold was reached on
the value: {value}");
    }

}
namespace Lab11Models;
```

```
public delegate void ValueHandler(int value);
```

ШІ використовувався тільки для ресьорчу.